

Maritime CargoService: Sprint 2

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 18, 2026

1. Introduction

Il punto di partenza di questo sprint è il prototipo realizzato nello Sprint 1, documentato al seguente [link](#)^o.

Lo Sprint 1 ha prodotto un prototipo eseguibile del **cargoservice** in cui il ciclo principale di carico viene gestito tramite attori QAK, con componenti simulati per IOPort, sonar, LED e markerdevice, e con il **cargorobot** modellato come wrapper del servizio **robotsmart26**.

L'architettura finale dello Sprint 1, riportata di seguito, costituisce il riferimento di partenza per lo Sprint 2.

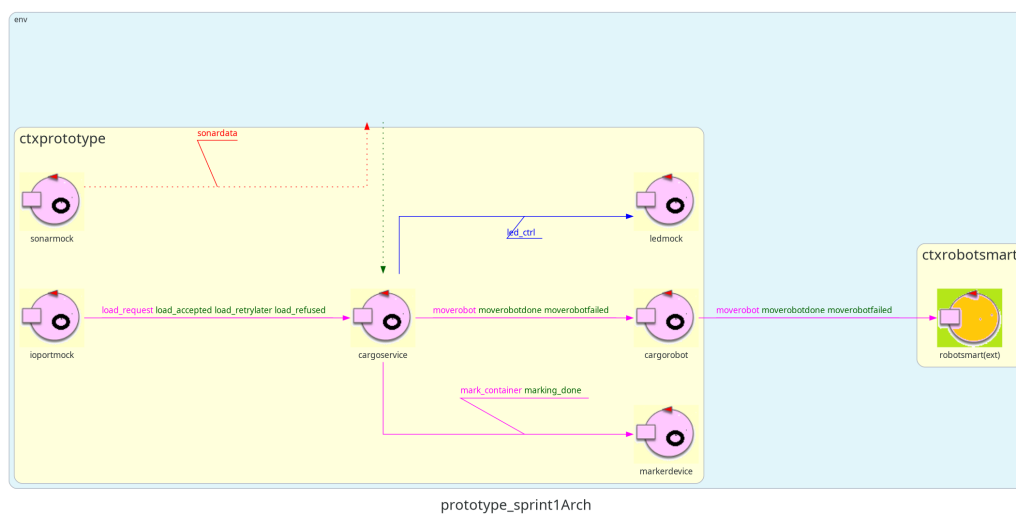


Figure 1: Architettura finale definita nello Sprint 1.

1.1. Goal dello Sprint 2

Il goal dello Sprint 2 è quello definito nella pagina di sintesi dello Sprint 1.

L'obiettivo dello Sprint 2 è evolvere il prototipo sviluppato sostituendo progressivamente i componenti simulati con le rispettive implementazioni reali:

- IOPort come Web GUI;
- sonar e LED collegati al PicoW;
- completare la gestione dello stato **Out of service** integrando il sonar reale, implementando la verifica della misura e l'interruzione/ripresa del movimento del cargorobot.

Si stima una durata di circa **30 ore** complessive di lavoro, corrispondenti a circa 10 ore per ciascun componente del gruppo.

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o.

Per comodità si riporta anche l'allegato con i requisiti forniti dalla committente.

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the *sonar sensor* measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the *sonar*).

Figure 2: Requisiti forniti dalla committente.

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della hold riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della hold.
- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{FREE}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.

- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold;
 - il messaggio “**Service working**” quando tutto sta procedendo correttamente;
 - il messaggio “**Out of service**” se il sensore sonar misura la distanza del container dal sonar stesso $D > D_{FREE}$ per almeno 3 secondi.

3. Requirement analysis

L'analisi dei requisiti è stata svolta nello [Sprint 0°](#) e approfondita nello [Sprint 1°](#). Tali risultati vengono assunti come validi anche per lo Sprint 2.

In particolare, per lo Sprint 2 restano centrali le seguenti conclusioni:

- il **cargoservice** Resta l'orchestratore del ciclo di carico;
- la **hold** rappresenta lo stato interno degli slot e delle posizioni rilevanti;
- il **cargorobot** viene trattato come servizio reattivo, tramite wrapper (SPECIFICA PATTERN) verso **RobotSmart26**;
- **sonar** e **LED** devono essere integrati come dispositivi reali collegati al PicoW;
- l'**IOPort** deve essere realizzato come Web GUI;
- lo stato **Out of service** deve bloccare l'accettazione di nuove richieste e interrompere il movimento del robot, con ripresa al ritorno dello stato **Service working**.

Dai requisiti si comprende come sul display dell'IoPort bisogna mostrare, oltre ai messaggi del display attualmente modellati come “Response” di QAK, lo stato corrente della Hold, che attualmente è auto-contenuto dal POJO. Bisognerà quindi riflettere su dei meccanismi di invio di aggiornamenti dello stato dalla Hold alla GUI dell'IoPort.

Sarà necessario inoltre, in base ai requisiti, decidere come e dove implementare la persistenza della misura per 3 secondi nelle fasi di ...

4. Problem analysis

Come definito precedentemente, bisogna sostituire progressivamente i componenti simulati con componenti accessibili attraverso tecnologie compatibili con la loro natura:

NOTA: SI HA A DISPOSIZIONE un ESP32 e non un PicoW

4.1. Osservabilità dello stato della Hold e considerazioni sulla Web GUI

Nello Sprint 1, l'IOPort era modellato come un **attore QAK mock** all'interno dello **stesso contesto della Hold**. Questa vicinanza gli permetteva di reagire direttamente ai messaggi scambiati sulla rete di attori.

Con l'evoluzione del sistema, l'IOPort diventerà una Web GUI eseguita in un browser esterno, introducendo così due problematiche che rendono i messaggi QAK nativi **inadatti** all'aggiornamento dell'interfaccia:

1. I browser **non comprendono** il protocollo di messaggistica QAK e non possono partecipare direttamente alla rete di attori.
2. Per rappresentare in tempo reale lo stato della Hold, la condizione Out of Service e l'occupazione dell'IOPort, la GUI necessita

di una vista consistente e sincronizzata del sistema in un determinato istante, anziché di una sequenza di messaggi da ricostruire ed elaborare lato client.

Per questo motivo lo stato dinamico del sistema andrà centralizzato in un'**unica rappresentazione** indipendente dall'implementazione interna degli attori e facilmente interpretabile dal client. In questo modo la Web GUI rimane disaccoppiata dalla logica applicativa, mentre il cargoservice continua a essere l'unico responsabile della gestione delle transizioni di stato e dell'aggiornamento della rappresentazione condivisa.

Si sceglie quindi il formato **JSON**, che può essere interpretato nativamente da JavaScript, tecnologia scelta per l'implementazione della **Web GUI**.

```
{
  "working": true,    // workingState: "Service working" | "Out of
service"
  "engaged": false,  // serviceState: "engaged" | "disengaged"
  "ioportBusy": false,
  "reservedSlot": null,
  "slots": {
    "slot1": "free",
    "slot2": "free",
    "slot3": "free",
    "slot4": "free",
    "slot5": "marker"
  }
}
```

4.2. Realizzazione dell'IOPort come Web GUI

Bisognerà quindi avere un server IOPort (middleware), denominato nel seguito **web server** o **IOPort server**, che

1. riceve i dati da cargoservice
2. Serve la pagina HTML CSS JAVASCRIPT
2. aggiorna la pagina in base agli aggiornamenti

Si sceglie di sviluppare questo server in javalin (?)

SNIPPET

4.2.1. Come ricevere dati da cargoservice?

ANALIZZA ALTRE SCELTE PROGETTUALI ad es. POLLING

Il runtime QAK permette a un attore di esporre nativamente il proprio stato come risorsa CoAP osservabile. Questa possibilità risulta adatta al problema poiché consente a un componente esterno di:

- recuperare lo stato corrente del sistema;
- osservare la risorsa;
- ricevere una notifica quando il suo contenuto viene aggiornato.
 - osservare la risorsa CoAP del **cargoservice**;
- tradurre gli aggiornamenti ricevuti in messaggi compatibili con il browser;
- ricevere dalla GUI le richieste dell'utente;
- inoltrare tali richieste al **cargoservice**.

Il **cargoservice** dovrà quindi esporre i dati JSON mediante la propria risorsa CoAP

SNIPPET

4.2.2. Come aggiornare dinamicamente la Web GUI?

Ricordiamo che l'IOPort richiesto dalla committente è costituito logicamente da:

- un pushbutton, utilizzato dal cliente per inviare una `load_request` ;
- un display, utilizzato per mostrare la risposta alla richiesta e lo stato corrente della hold.

Essa si limita a:

- inviare il comando associato alla pressione del pushbutton;
- visualizzare la risposta ricevuta;
- aggiornare il display quando cambia lo stato pubblicato dal **cargoservice**.

Non parlerà direttamente col cargoservice ma con quel middleware

Per i dati da “visualizzare”: L'aggiornamento dello stato non dovrebbe dipendere da interrogazioni periodiche effettuate dal browser, poiché il polling introdurrebbe richieste ripetute anche in assenza di cambiamenti. usiamo WebSocket

Per invece le richieste: usiamo richieste HTTP, si sceglie la POST

4.2.2.1. Invio della load_request

Quando il cliente preme il pulsante, la GUI invia una richiesta HTTP `POST` al server intermediario.

Il server traduce tale richiesta in una `load_request` indirizzata al **cargoservice** e attende una delle risposte già definite nello Sprint 1:

- `load_accepted` ;
- `load_retrylater` ;
- `load_refused` .

La risposta viene quindi restituita alla GUI e mostrata sul display.

Questa soluzione mantiene invariato il protocollo applicativo QAK già validato nello Sprint 1: HTTP viene utilizzato soltanto nel tratto browser-server, mentre l'interazione con il **cargoservice** continua a essere espressa mediante i messaggi del modello.

4.2.2.2. Aggiornamento del display

1. il server osserva la risorsa CoAP del **cargoservice**;
2. quando la risorsa cambia, il server riceve il nuovo JSON;
3. il server inoltra l'aggiornamento ai browser connessi tramite WebSocket;
4. la GUI aggiorna il display.

4.3. Integrazione del sonar reale

Nello Sprint 1 il sonar era rappresentato da **sonarmock**, che emetteva eventi di tipo **sonardata**. La logica del **cargoservice** dipende quindi dal contenuto delle misurazioni, ma non dalla concreta implementazione del dispositivo che le produce.

Nello Sprint 2 il sonar fisico è collegato al ESP32. Il software eseguito sul dispositivo deve:

- leggere periodicamente la distanza;
- rendere disponibili le misurazioni al sistema distribuito;
- rimanere indipendente dall'implementazione interna del **cargoservice**.

L'ESP32 e il sistema QAK operano su piattaforme differenti. È pertanto necessario utilizzare un protocollo interoperabile e sufficientemente leggero per un dispositivo IoT.

Si sceglie MQTT, basato sul modello publish/subscribe. L'ESP32 pubblica le misurazioni su un topic dedicato, mentre il sistema software si sottoscrive al medesimo topic e le traduce nel messaggio `sonardata` utilizzato dal **cargoservice**.

La corrispondenza tra topic MQTT e messaggio QAK deve essere configurata esplicitamente. In questo modo la logica applicativa del **cargoservice** può continuare a elaborare `sonardata` senza dipendere dal linguaggio o dalla piattaforma utilizzati dal dispositivo fisico.

Il sonar conserva quindi una responsabilità limitata:

- misurare la distanza;
- pubblicare la misura.

L'interpretazione applicativa della distanza rimane invece responsabilità del **cargoservice**.

4.4. Validazione temporale delle misure sonar

I requisiti non associano un cambiamento di stato a una singola misura istantanea. La presenza del container e il possibile guasto del sonar devono essere riconosciuti soltanto quando la relativa condizione permane per almeno tre secondi.

Occorre quindi distinguere:

- $D < D_FREE/2$: possibile presenza del container;
- $D > D_FREE$: possibile condizione di guasto;
- valori intermedi: assenza di una delle due condizioni precedenti.

Una singola misura non è sufficiente a produrre una transizione. La condizione deve essere confermata attraverso misure consecutive per l'intervallo temporale richiesto.

La responsabilità della validazione temporale può essere collocata:

- sul ESP32, che pubblica soltanto condizioni già validate;
- nel sistema QAK, che riceve tutte le misurazioni e gestisce il tempo di permanenza.

Per mantenere il dispositivo focalizzato sull'acquisizione dei dati e centralizzare nel **cargoservice** le decisioni applicative, si sceglie di trasmettere le misurazioni grezze e di effettuare la validazione temporale lato sistema software.

Il **cargoservice** deve quindi mantenere separatamente:

- l'istante di inizio della condizione $D < D_FREE/2$;
- l'istante di inizio della condizione $D > D_FREE$;
- l'eventuale annullamento del conteggio quando la condizione non è più verificata.

Solo dopo il superamento dell'intervallo stabilito viene aggiornato lo stato del sistema.

4.5. Integrazione del LED reale

Nello Sprint 1 il **cargoservice** inviava a **ledmock** il dispatch:

```
Dispatch led_ctrl : ledCmd(CMD)
```

Il comando rappresentava le operazioni logiche **on** , **off** e **blink** .

Nello Sprint 2 il LED è fisicamente collegato al ESP32. Anche in questo caso è necessario evitare che il **cargoservice** dipenda dai dettagli hardware del dispositivo.

Il messaggio **led_ctrl** viene pertanto mantenuto come interfaccia logica. Un componente di integrazione riceve il comando e lo pubblica su un topic MQTT dedicato al LED. Il software sul ESP32 si sottoscrive al topic e traduce il valore ricevuto nell'operazione hardware corrispondente.

La catena logica diventa quindi:


```
cargoservice -> led_ctrl -> adattatore MQTT -> ESP32 -> LED
```

In questo modo:

- il **cargoservice** continua a utilizzare lo stesso comando definito nello Sprint 1;
- la gestione elettrica del LED rimane confinata sul ESP32;
- il componente fisico può essere sostituito senza modificare la logica applicativa.

4.6. Gestione dello stato Out of service

Nello Sprint 1 il **cargoservice** aggiornava la variabile `ServiceWorking` sulla base degli eventi sonar, ma non interrompeva un movimento già in corso.

Nello Sprint 2 il comportamento viene completato in accordo con il chiarimento fornito dalla committente:

- quando la condizione `D > D_FREE` persiste per almeno tre secondi, il sistema entra nello stato **Out of service**;
- durante tale stato non vengono accettate nuove richieste;
- se il robot è in movimento, il piano deve essere interrotto;
- quando il sonar torna a misurare `D <= D_FREE`, il sistema ritorna nello stato **Service working**;
- se un movimento era stato interrotto, deve essere ripreso.

La gestione richiede di distinguere almeno due situazioni:

1. il sistema entra in **Out of service** mentre non è in corso alcun movimento;
2. il sistema entra in **Out of service** durante l'esecuzione di una procedura di movimentazione.

Nel primo caso è sufficiente aggiornare lo stato operativo e impedire l'accettazione di nuove richieste.

Nel secondo caso il **cargoservice** deve inoltre richiedere l'interruzione del movimento e ricordare che esiste una procedura sospesa. Al ripristino del servizio, il movimento viene ripreso secondo le operazioni offerte da **RobotSmart26**.

L'ingresso nello stato **Out of service** non deve:

- liberare automaticamente lo slot riservato;
- riportare il sistema nello stato `disengaged`;
- annullare la procedura di carico.

Si tratta infatti di una sospensione temporanea e non del fallimento definitivo dell'operazione.

4.7. Gestione coerente dello stato della hold

La prenotazione di uno slot e la sua effettiva occupazione rappresentano due momenti distinti.

Quando una `load_request` viene accettata, lo slot viene riservato affinché non possa essere assegnato a un'altra richiesta. Esso può però essere considerato fisicamente occupato soltanto dopo che il robot ha completato il deposito.

È quindi opportuno distinguere logicamente almeno i seguenti stati:

- libero;
- riservato;
- occupato.

Questa distinzione permette alla Web GUI di mostrare uno stato più fedele della hold e rende esplicito cosa debba accadere nei casi di timeout, sospensione o fallimento del movimento.

In particolare:

- in caso di timeout prima del deposito, lo slot riservato torna libero;
- durante una sospensione **Out of service**, lo slot rimane riservato;
- dopo il deposito completato, lo slot diventa occupato;
- in caso di fallimento del robot, lo slot non viene liberato automaticamente.

4.8. Evoluzione dell'architettura

Rispetto allo Sprint 1 vengono introdotti i seguenti elementi:

- una Web GUI che realizza l'IOCome posso evolvere lo sprint1 naturalmente nello sprint2?Port;
- un server intermediario per HTTP, WebSocket e osservazione CoAP;
- un broker MQTT per la comunicazione con L'ESP32;
- il software sul ESP32 per il sonar e il LED;
- un componente di adattamento tra MQTT e i messaggi QAK;
- la pubblicazione dello stato del **cargoservice** come risorsa CoAP osservabile.

Il **cargoservice** rimane l'orchestratore del ciclo di carico e la **Hold** rimane la sorgente dello stato degli slot. I nuovi componenti non trasferiscono altrove la logica applicativa, ma rendono possibile l'interazione con browser e dispositivi fisici.

L'evoluzione preserva quindi le interfacce logiche introdotte nello Sprint 1:

- `load_request` e relative reply per l'IOPort;
- `sonardata` per le misurazioni del sonar;
- `led_ctrl` per il controllo del LED;
- `moverobot` per la movimentazione;
- `mark_container` per la marcatura.

Le principali modifiche riguardano il trasporto dei messaggi e l'osservabilità dello stato, non il significato delle interazioni applicative già validate.

5. Project

5.1. Struttura del progetto

Nota: Da completare con la struttura finale dei moduli e con i riferimenti ai sorgenti implementati nello Sprint 2.

5.2. Implementazione dell'IOPort

Nota: Da completare.

5.3. Implementazione dei dispositivi reali

Nota: Da completare.

5.4. Implementazione della gestione Out of service

Nota: Da completare.

6. Test plans

Nota: Da completare con i test funzionali e di integrazione previsti per lo Sprint 2.

6.1. Test IOPort

Nota: Da completare.

6.2. Test sonar e LED

Nota: Da completare.

6.3. Test Out of service e ripresa del robot

Nota: Da completare.

7. Deployment

Il deployment del prototipo relativo allo **Sprint 2** richiede l'orchestrazione di diversi contesti distribuiti su nodi logici o fisici distinti (motore QAK, server intermedio Web/CoAP, ambiente di simulazione WEnv e broker MQTT).

7.0.1. Prerequisiti di Sistema

Per l'esecuzione end-to-end del sistema è necessaria la presenza dei seguenti strumenti:

- **JDK**;
- **Docker** e **Docker Compose**;
- **Browser Web**.

7.0.2. Avvio Automatizzato

Al fine di semplificare la fase di testing ed evitare conflitti di binding sulle porte di rete, all'interno della cartella `Scripts_Avvio` è stata predisposta una suite di script automatici sia per ambienti **Linux/macOS** che **Windows**.

Gli script eseguono una pulizia preventiva delle porte (`8020` , `8050` – `8053` , `8085` , `8086` , `8090`) e dei container Docker residuali, per poi avviare i 7 processi in sequenza ordinata con i corretti tempi di sincronizzazione:

1. **Ambiente Virtuale WEnv e Broker MQTT** (`docker compose -f unibobasic26.yaml up`);
2. **Servizio Base e Pathfinder** (`robotsmart26` su porta `8020`);
3. **Orchestratore Centrale e Risorsa CoAP** (`cargoservice` su porta `8050`);
4. **Wrapper Trasportatore QAK** (`robot` su porta `8053`);
5. **Contesto Cliente e LedMock** (`customer` su porta `8051`);
6. **Server Intermedio Facade Web GUI / CoAP Observer** (`IOPortServer` su porta `8086`);
7. **Simulatori Hardware Sonar e Marker** (`devices` su porta `8052`).

7.0.2.1. Esecuzione su Linux e macOS

Aprire un terminale nella cartella radice del progetto e avviare lo script dedicato:

```
cd Progetto/Scripts_Avvio ./start_all_sprint2.sh
```

Per arrestare e pulire l'intero sistema al termine delle prove:

```
./stop_all_sprint2.sh
```

7.0.2.2. Esecuzione su Windows

Da Prompt dei comandi (oppure effettuando un doppio clic su Esplora Risorse):

```
cd ProgettoScripts_Avvio start_all_sprint2.bat
```

Lo script aprirà automaticamente 7 finestre del Prompt dei comandi per i rispettivi contesti. Per arrestare l'esecuzione:

```
stop_all_sprint2.bat
```

7.0.3. Avvio Manuale via Gradle

Qualora si preferisca avviare e monitorare singolarmente i vari contesti (ad esempio per attività di debug o profilazione), è possibile eseguire i seguenti comandi da terminali separati:

1. WEnv & MQTT (da `sprint2/robotsmart26/yamls`)

```
docker compose -f unibobasic26.yaml up
```

2. Robot base (da `sprint2/robotsmart26`)

./gradlew run

3. CargoService (da sprint2/prototype/cargoservice)

./gradlew run

4. CargoRobot (da sprint2/prototype/robot)

./gradlew run

5. Customer / Led (da sprint2/prototype/customer)

./gradlew runCustomer

6. IOPortServer Web Facade (da sprint2/prototype/customer)

./gradlew runIOPortServer

7. Devices / Sonar (da sprint2/prototype/devices)

./gradlew run

7.0.4. Verifica e Interazione con il Sistema

Al termine della sequenza di avvio, l'interfaccia utente è accessibile da browser tramite due endpoint principali:

- <http://localhost:8090>[°] : per monitorare visivamente i movimenti del robot all'interno dell'ambiente di simulazione tridimensionale (WEnv);
- <http://localhost:8086>[°] : per accedere alla Web GUI della IOPort. Da questa pagina l'operatore può:
 - inviare richieste di carico premendo il pulsante LOAD;
 - osservare l'esito della richiesta (accepted , retrylater , refused);
 - monitorare in tempo reale (tramite notifiche push WebSocket da osservatore CoAP) la transizione di stato degli slot della stiva (free , reserved , occupied) e lo stato operativo del servizio (Service working vs Out of service).

8. Maintenance

Nota: Da completare con eventuali note di manutenzione, limiti noti e attività rinviate agli sprint successivi.

9. Pagina di sintesi

9.1. Architettura finale dello Sprint 2

Nota: Da completare con l'architettura finale prodotta nello Sprint 2.

9.2. Goal dello sprint successivo

Nota: Da completare al termine dello Sprint 2.

10. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340